

Smart data Masking Function Requirements/Possibilities

Carlos Poses

Table of contents

0.1	Problem	1
0.2	Requirement 1	2
0.2.1	Example	2
0.2.2	Expressing the problem using an equation	2
0.2.3	Three scenarios where requirement 1 is not satisfied	2
0.2.4	Conclusion	4
0.3	Requirement 2: For a masked value, it is impossible to tell anything from the value other than it is less than the threshold	4
0.3.1	Problem	4
0.3.2	Three scenario	5
0.4	Possible solutions	8
0.4.1	Possible solutions	8

0.1 Problem

- In tables that display the counts of categorical variables, we want to mask values for categories whose count is less than 5 (or some other threshold T).

I'll call 'naive masking' simply changing values less than a threshold by a string (i.e., '<5'). We need to go further than this, though. To properly implement masking, there are two requirements:

- **Requirement 1:** We need to make sure the value of the masked value can't be computed from the other values and the totals.
- **Requirement 2:** We need to make sure that it's not possible to exclude some values in the interval $[1, T-1]$. I.e., for each masked category, all values in the interval need to be possible.

In this doc: First, I explain when naive masking does not work. This results in some rules that can (probably) be implemented in a function that checks every table to check all values are properly masked. I also discuss when naive masking works, but possible solutions are small so that naive masking may still be considered ineffective. Second, I discuss what to do when naive masking does not working but we still want to give as much info as possible (and the category totals will be given). ## Requirement 1

0.2 Requirement 1

Simply put, naive masking does not work whenever the masked values (the unknowns) are computable from known values (totals and other values). Equivalently: whenever the problem can be represented as an equation for which there's an unique solution.

0.2.1 Example

```
cat_value <- c(3, 97, 100)
total <- sum(cat_value)
total
```

```
[1] 200
```

Masking the value 3 (e.g., substituting it by ' ≤ 5 ') is not enough if we know the other values and the total: $200 - 97 - 100 = 3$.

0.2.2 Expressing the problem using an equation

For each category, the problem can be represented with the equation:

$$x + y + \dots + z = N$$

where $x, y, \dots, z$ are known integers representing category counts and N is the total. We denote masked values (values whose count is less than the threshold) with superscript M (e.g., x^M). Masking won't work if after substituting one value by its counterpart, the equation has only one solution.

0.2.3 Three scenarios where requirement 1 is not satisfied

I came up with 3 scenarios when the exact value is computable from the other counts and the totals.

0.2.3.1 Scenario 1

Masking only one value without masking another value (or the total)

$$x + y + \dots + z = N$$

If we mask one value (x^M), this turns into an linear equation with one unknown and thus an unique solution.

$$x^M = N - y - \dots - z$$

(This is the case represented by the first example)

0.2.3.2 Scenarios 2 and 3: Masking two or more values, but unique solutions still exist

In general an equation with two or more unknowns has no solution. But given the restrictions (all values are positive integers), there are two special cases where there's only one possible solution:

0.2.3.2.1 When all original masked values are one, because one can still work it out from the total.

For instance, we mask two values.

$$x^M + y^M + \dots + z = N$$

If $x^M = y^M = 1$ then

$$x^M + y^M = N - \dots - z = 2$$

Even if you don't know what the original x^M and y^M values are, if you know \$ N - \dots - z \$ and the number of masked values (# M), the only solution is that all values are 1. This is general always the case when the result of subtracting unmasked values from the total is equal to the number of masked values.

0.2.3.2.2 When all masked values are equal to the highest possible value of the masked values

Say threshold is less than 5.

$$x^M + y^M + \dots + z = N$$

If $x^M = y^M = 4$ then

$$x^M + y^M = N - \dots - z = 8$$

Again there's only one solution for the individual values of x^M and y^M . So this is equivalent to not masking

0.2.4 Conclusion

Naive masking won't work if you can specify the counts as an equation, and it is possible to work out the unknowns (masked values), i.e., the equation has unique solutions.

Possible solution: Try to solve the system of equations with some R library (or base R) and proper restrictions (positive integers). If there's unique solution to the there's no masking.

NB: I am not 100% sure there're more than the 3 scenarios where naive masking will not work, but I think framing the problem as 'is there an unique solution to this equation, where the masked values are the unknown?' should cover every possibly scenario. As long as we find a way to always answer this question (given restrictions on possible solutions), it should be fine.

0.3 Requirement 2: For a masked value, it is impossible to tell anything from the value other than it is less than the threshold

0.3.1 Problem

- Typically when we mask, we say a value is less than 5. What we want is to give no other information than that value is an integer in the interval [1,4]. For all categories, the values 1, 2, 3 and 4.
- Sometimes when applying naive masking, it's impossible to know the exact value, but it is still possible to exclude some values from the interval.

0.3.2 Three scenario

Depending on the information available from the sum of the unmasked values and the total, you can classify all cases in 3 scenarios.

0.3.2.1 Scenario 1: You can always exclude some values

- Whenever this is true, you know for sure some values included in the interval are impossible.

$$N - w - \dots z < T + \#M - 2$$

where T is the threshold (i.e. 5 in less than 5), $\#M$ is the number of ‘Naively masked’ variables, N is the total and $w, \dots, z$ are the counts for other variables.

0.3.2.1.1 Example

```
cat_value <- c(2, 1, 97, 100)
total <- sum(cat_value)
total
```

```
[1] 200
```

If our threshold is 5, we would mask the values 2 (let’s call it x^M) and 1 (let’s call it y^M). But then we know

$$200 - 97 - 100 = 3$$

or

$$N - z - w = x^M + y^M$$

Even though we want to say that the value for both x and y is < 5 (i.e., between 1 and 4), in this example you can tell that for x (or y) it is either 2 or 1. It’s impossible to tell the exact value, because there’s no unique solution. But we know the value is neither 3 nor 4.

0.3.2.2 Scenario 2: You are not giving away any information

$$N - w - \dots z = T + \#M - 2$$

```
cat_value <- c(2, 3, 97, 100)
total <- sum(cat_value)
total
```

```
[1] 202
```

```
# we would mask 2 and 4, let's call them x and y
total - 100 - 97
```

```
[1] 5
```

You know one of x or y is not 4 (because both can't be 5. Actually both can't be 3 either, or one 4 and another 3, etc). But you are saying nothing about the values: x could be 5, 4, 3, 2 or 1. Same for y.

You reveal some info: both masked are not close to the threshold. But you can't tell which one is closer, and all values below the threshold are possible for both counts. This is the info you are giving: *at least one of the values is not equal to 4 (or equal to $T - 1$ in general)*. But you can't tell for *any specific value* whether it is exactly equal to 5 or not.

0.3.2.2.1 Potential problem

This strategy might seem ok, but it scales a bit badly. Say that, naively, you would mask 6 values.

```
cat_value <- c(2,1,1,2,3, 4, 97, 100)
total <- sum(cat_value)
total
```

```
[1] 210
```

```
total - 100 - 97
```

```
[1] 13
```

Here you know that the sum of the 6 masked values is 13. All values are 'well masked', i.e., you can't tell for any specific value whether it's 5,4,3,2, or 1. But you are giving away the info that most values are not 5, or 4, or 3. It can be a bit problematic in some edge cases (e.g. there are 10 masked values that add up to 14, you know many values are 1).

0.3.2.3 Scenario 3: You may or may not give away some info

$$N - w - \dots z > T + \#M - 2$$

In this case, sometimes you may not be able to exclude any values, but sometimes you will.

0.3.2.3.1 Example

```
cat_value <- c(2, 4, 92, 100)
total <- sum(cat_value)
total
```

```
[1] 198
```

$$198 - 100 - 192 = 6$$

We would mask two values, and we know their sum is equal to 6. If the threshold is 5 (all values are in the interval $[1,4]$), that makes it impossible that either value is 1.

0.3.2.4 Conclusion

You need to make sure that all possible values in the interval are possible for all values.

I think this is equivalent to having an equation with multiple solutions, where at least one solution per masked count is equal to all values in the interval. E.g., there's a solution where $x^M = 1$, one where $x^M = 2$, one where $x^M = 3$... and check that for every masked value and possible interval.

Possible way to solve this: frame the problem as an equation where masked variables are unknown, and ask 'Is there at least one solution where the unknown takes the value 1, one where the unknown takes the value 2, one where the unknown takes the value 3... for all possible values and unknowns'. Should be possible to program this, and given restrictions (only positive integers), it shouldn't be too computationally expensive. It may be a bit of a challenge to program this, though.

0.4 Possible solutions

So far, I have focused on when masking works: under which conditions requirements 1 and 2 are satisfied. The main takeaway is that both requirements can be framed as ‘are solutions to this equation possible’. With this, we can work an algorithm of the form:

1. Apply naive masking
2. Check after this, requirements 1 and 2 are satisfied.
3. If TRUE, table is fine. Naive masking is enough (or there’s no masking). If FALSE, table is not fine. We need to do something else.

0.4.1 Possible solutions

If naively you would mask only one value, one solution is to interval-mask an additional value (e.g., the second smallest value). It’s easier to reason with an example:

```
cat_value <- c(3, 97, 100)
total <- sum(cat_value)
total
```

```
[1] 200
```

Here, naive masking is not enough. If you mask 3, you know the value from the totals. A possible solution is to ‘interval-mask’, e.g.,

```
c('1-4', '99-95', 100)
```

```
[1] "1-4" "99-95" "100"
```

As far as I know, this solution can be expanded to all scenarios. The algorithm is. 1. Naive mask 2. If requirements are not satisfied, interval-mask another value. Assuming the interval of masked variables is $[1,4]$, the interval for the additionally interval-masked value will be:

$$[N - \sum Unmasked - \# * 1, N - \sum Unmasked - \# * 4]$$

NB: Choosing the value to smart-mask can be tricky. I would think it’s the smallest unmasked value, but if it’s close to the threshold (e.g., threshold is 6, interval-masked value is 6, the interval could be for instance $[4,8]$, which tells you is definitely not 4 (else it would be masked!)). Choosing this smartly needs some more thought.

0.4.1.1 Other solutions

Intuitively I chose this solution because I think it's the solution where we leave as much values unaltered as possible. I haven't explored other solutions yet. Possible candidates are:

- Rounding/adding noise: the problem is for this to work, we change every value. We lose a lot of precision.
- Differential privacy (honestly not sure what this is)
- Merging categories: could be an option too?
- Exploring what's there: I found a few resources, of which the most important ones are:
- This very recent guide on statistical output checking by Health Data & Analytics Research Environments UK programme. <https://zenodo.org/records/10282526>
- This R package they seem to be creating that creates common analysis (e.g., tables or models) with in-built statistical output control.